

IEEE-CIS Financial Fraud Detection Engine

A-to-Z Technical Master Breakdown & Technical Interview Handbook

1. Executive Project Summary

Interview Pitch ("Tell me about your project" — 1-Minute Answer):

"In my project, I engineered an end-to-end Financial Fraud Detection Engine using XGBoost on the IEEE-CIS benchmark dataset (1.1 Million rows, 430+ features). The main challenge was extreme class imbalance (3.5% fraud) and system memory constraints. I built a disk-cached ETL pipeline that downcasted data types, extracted time-delta features, and pruned 159 redundant sparse columns. Using 3-Fold Stratified K-Fold CV with XGBoost's histogram-based tree algorithm (`tree_method='hist'`), the model achieved an Out-Of-Fold (OOF) ROC-AUC of 0.9421 and a Fraud Precision of 0.92 in under 4 minutes while keeping RAM below 1.5 GB."

End-to-End Pipeline Architecture

Stage	Implementation Details	Key Technical Highlight
1. Data Ingestion	Chunked CSV reading (50k rows/batch), float64 to float32, int64 to int8/int16 downcasting.	50% RAM reduction during load
2. Disk Caching	Sequential train/test processing, cached to disk via joblib.	Train & test never sit in RAM together
3. Feature Engineering	Extracted DT_hour, DT_day_of_week, DT_W, DT_M; Log(Amt+1) scaling; 43 label encodings.	Cyclic time-delta capture
4. Feature Selection	Pruned 159 Vesta columns with >= 50% missing values (437 -> 278 features).	Matrix memory dropped to 626 MB
5. Model Training	3-Fold Stratified K-Fold CV using low-level xgb.DMatrix and <code>tree_method='hist'</code> .	Eliminated C++ memory allocation errors
6. Test Inference	Chunked Booster predictions in 100k batches across 3 fold models.	506,691 test rows processed in seconds

2. Technical Deep-Dive & Performance Results

Metric	Achieved Value	Technical & Business Significance
OOF ROC-AUC	0.94208	Exceptional separation between Fraud and Legitimate distributions (Fold 1: 0.944, Fold 2: 0.939, Fold 3: 0.942).
Precision (Fraud)	0.92	Out of 100 transactions flagged as fraud, 92 are actual fraud. Minimizes false declines for customers.
Recall (Fraud)	0.48 (at 0.5 thresh)	Catches 48% of total fraud at 0.5 threshold. Lowering threshold (e.g. 0.15) increases recall to 80%+.
Overall Accuracy	98.0%	High overall accuracy; dataset contains 96.5% legitimate transactions.
Training Latency	3.84 mins	Fast training on standard CPU using histogram binning (tree_method='hist').

3. Technical Interview Question & Answer Bank

Category A: Fundamental & Architectural Questions

Q1: Why choose XGBoost over Deep Learning (Neural Networks) or Random Forest?

Answer: On tabular data with high missingness and mixed types (categorical + continuous), Gradient Boosted Decision Trees (GBDTs) consistently outperform Deep Learning models. XGBoost handles missing values natively without imputation artifacts, and tree_method='hist' enables fast execution on CPU with minimal RAM footprint.

Q2: What is the difference between tree_method='hist' and tree_method='exact'?

Answer: 'exact' evaluates every unique value as a split point, costing $O(N * D)$ per node with high memory overhead. 'hist' discretizes continuous features into fixed bins (max_bin=128), reducing split finding to $O(\text{bin_count} * D)$. This cuts memory usage by >70% and speeds up training by 5x with zero loss in ROC-AUC.

Q3: What loss function is used, and how does XGBoost optimize it?

Answer: Binary Cross-Entropy (Log-Loss): $L(y, p) = -[y \log(p) + (1-y) \log(1-p)]$. XGBoost computes a 2nd-order Taylor expansion of the loss, using 1st order gradients (g_i) and 2nd order Hessians (h_i) to determine optimal split gains and leaf node weights.

Category B: Data & Preprocessing Questions

Q4: How did you handle the extreme Class Imbalance (3.5% Fraud)?

Answer: 1) Used 3-Fold Stratified K-Fold CV to preserve the 3.5% fraud ratio in every fold. 2) Selected ROC-AUC and Precision-Recall as core evaluation metrics instead of Accuracy. 3) Post-training probability threshold tuning to adjust Precision vs. Recall according to business cost functions.

Q5: How did you resolve Out-Of-Memory (OOM) errors during local execution?

Answer: 1) Disk-cached train and test DataFrames separately using joblib so they were never in RAM simultaneously. 2) Downcasted types (float64 -> float32, int64 -> int8/int16). 3) Pruned 159 sparse Vesta columns with $\geq 50\%$ missing values (dropping matrix size to 626 MB). 4) Passed C-contiguous float32 NumPy matrices to low-level xgb.DMatrix to eliminate intermediate memory copies.

Category C: Deployment & System Design Questions

Q6: How would you deploy this model for low-latency real-time inference?

Answer: Package the feature transformations and exported XGBoost Booster JSON model into a FastAPI microservice containerized with Docker. Single transaction JSON payloads are transformed into a 1D NumPy float32 array, passed to `booster.predict()`, returning a fraud probability in under 5 milliseconds.

Q7: How would you scale inference for 10,000 requests/sec?

Answer: 1) Convert model to Treelite or ONNX Runtime to compile decision trees into native C instructions (3-5x faster). 2) Use Redis in-memory cache for O(1) user profile and categorical lookups. 3) Implement dynamic micro-batching (50 requests/batch) for parallel execution.

Category D: Trick & Edge-Case Questions

Q8: What if the dataset grows by 10x (e.g. 50 Million rows)?

Answer: 1) Migrate pipeline to PySpark or Dask-XGBoost distributed cluster. 2) Use XGBoost's external memory `QuantileDMatrix` iterator to stream Parquet chunks from S3/disk. 3) Utilize Feast or Hopsworks Feature Store to decouple feature computation from model training.

Q9: How will you detect and handle Concept Drift in production?

Answer: 1) Data Drift: Track Population Stability Index (PSI) and Wasserstein distance on key features (e.g. TransactionAmt). 2) Performance Drift: Monitor 7-day rolling Precision/Recall against chargeback logs. 3) Retraining: Trigger automated retraining (Airflow / MLflow) weekly or when $PSI > 0.25$.